

Part B – Manage Filtering Laws

A dialog opened from a button beside Apply: the laws of the selected aircraft on top, a detail form below, and Create, Update and Delete. `filtering_law` and `contextual_parameter` already exist, so the only schema change is the one flag in Step 1 – and that one needs a decision first.

What the code already gives us

Found	Means
The enum <code>com.eurocopter.mgs.datamodel.health.common.ContextualParameter</code> lists every allowed parameter name	The parameter dropdown is populated from <code>values()</code> . Nothing hardcoded.
<code>FilteringLaw.contextualParameters</code> is cascade ALL with <code>orphanRemoval</code>	Saving the law saves its parameters; removing one from the set deletes its row. No separate parameter DAO needed.
<code>ContextualParameter.minValue</code> / <code>maxValue</code> are primitive double, <code>thresholdOfVariabilityValue</code> is a Double	Min and max are mandatory; stability is optional, which matches HELIONIX requiring it and M'ARMS not.
<code>BatchMarkerCreationDialog</code> extends <code>Dialog</code> with a separate composite and a <code>FormToolkit</code>	That is the house style. <code>TitleAreaDialog</code> is not used anywhere in MGS, so the new dialog follows this instead.

Two things to settle before writing code.

1. Greying imported laws. The requirement says laws that came from the aircraft configuration file are read-only. `FilteringLaw` has no field recording where it came from, so nothing today can tell an imported law from a user-created one. Step 1 adds a flag. Confirm with Benoit that a new column is acceptable; if not, the alternative is to treat every law as editable and rely on the in-use checks alone.

2. Renaming a law is dangerous. `indicator_configuration.filtering_law` holds the law *name* as a string, not a foreign key. Renaming a law therefore silently orphans every indicator pointing at it. Step 9 forbids renaming a law that is in use, which is the safe reading. If the client wants rename to work, the update must also rewrite those rows – ask before assuming.

Server

Step 1 — FilteringLaw – one new field

mgs-jpa-model/.../datamodel/health/monitoring/FilteringLaw.java

Records whether the law came from the configuration file. Defaults to false so every existing row stays editable, and the import path sets it true.

```
@Column(name = "ORIGINAL")
private boolean original;

public boolean isOriginal() {
    return original;
}

public void setOriginal(boolean original) {
    this.original = original;
}
```

Step 2 — DDL – run in pgAdmin

One column, defaulted so existing rows are valid.

```
ALTER TABLE public.filtering_law
ADD COLUMN original boolean NOT NULL DEFAULT false;
```

Step 3 — The configuration import – set the flag

Find where laws are created during import: search *.java for new FilteringLaw(. In the delegate method that builds them (createFilteringLawConfiguration or similar), add one line.

```
filteringLaw.setOriginal(true);
```

Laws created from the new dialog leave it false, so only imported ones are greyed.

Step 4 — ContextualParameterDto – new

mgs-services-interfaces/.../service/health/common/dto/ContextualParameterDto.java

Mirrors the entity. Min and max are Double here rather than double, so an empty field arrives as null and can be rejected with a message instead of defaulting to zero.

```
public class ContextualParameterDto implements Serializable {
    private static final long serialVersionUID = 1L;

    private String id;
    private String name;
    private Double minValue;
    private Double maxValue;
    private Double thresholdOfVariabilityValue;
    // getters and setters
}
```

Step 5 — FilteringLawDto – three more fields

mgs-services-interfaces/.../service/health/common/dto/FilteringLawDto.java

It currently carries only id and name. The dialog needs the parameters and the two read-only reasons.

```
private List<ContextualParameterDto> parameters = new ArrayList<>();
private boolean original; // came from the configuration file
private boolean inUse; // referenced by an indicator configuration or a deviation

public List<ContextualParameterDto> getParameters() { return parameters; }
public void setParameters(List<ContextualParameterDto> p) { this.parameters = p; }
public boolean isOriginal() { return original; }
public void setOriginal(boolean original) { this.original = original; }
public boolean isInUse() { return inUse; }
public void setInUse(boolean inUse) { this.inUse = inUse; }
```

A law is read-only in the dialog when isOriginal() or isInUse() is true. Keeping them separate lets the error message say which reason applies.

Step 6 — FilteringLawDao – new

mgs-services-impl/.../service/health/common/ejb/FilteringLawDao.java

Laws hang off the health configuration, so create and delete go through it. HealthDao already exposes `getLatestConfiguration`, so inject it rather than duplicating the lookup.

```
package com.eurocopter.mgs.service.health.common.ejb;

@Local
@Stateless
public class FilteringLawDao {

    @EJB
    private HealthDao healthDao;

    @PersistenceContext
    private EntityManager manager;

    /** Every law of the aircraft, with its parameters loaded. */
    public List<FilteringLaw> findAll(String aircraftId) throws EntityNotFoundException {
        List<FilteringLaw> laws = new ArrayList<>()
        healthDao.getLatestConfiguration(aircraftId).getFilteringLaws();
        for (FilteringLaw law : laws) {
            law.getContextualParameters().size(); // force load before detaching
        }
        return laws;
    }

    public FilteringLaw find(String lawId) {
        FilteringLaw law = manager.find(FilteringLaw.class, lawId);
        if (law != null) {
            law.getContextualParameters().size();
        }
        return law;
    }

    public void create(String aircraftId, FilteringLaw law) throws EntityNotFoundException {
        healthDao.getLatestConfiguration(aircraftId).getFilteringLaws().add(law);
        manager.persist(law);
    }

    public void delete(String aircraftId, String lawId) throws EntityNotFoundException {
        FilteringLaw law = manager.find(FilteringLaw.class, lawId);
        if (law == null) {
            return;
        }
        healthDao.getLatestConfiguration(aircraftId).getFilteringLaws().remove(law);
        manager.remove(law);
    }

    /** True when any indicator configuration of this aircraft points at the law name. */
    public boolean isUsedByConfiguration(String aircraftId, String lawName)
        throws EntityNotFoundException {
        for (HealthMechanicalComponentConfiguration mcc : healthDao.getLatestConfiguration(aircraftId)
            .getMechanicalComponentConfigurations().values()) {
            for (IndicatorConfiguration ic : mcc.getIndicatorConfigurations().values()) {
                if (lawName.equals(ic.getFilteringLaw())) {
                    return true;
                }
            }
        }
        return false;
    }

    /** True when any deviation, active or acknowledged, references the law. */
    public boolean isUsedByDeviation(String lawId) {
        Long count = manager.createQuery(
            "SELECT COUNT(d) FROM IndicatorDeviation d WHERE d.filteringLaw.id = :lawId",
            Long.class)
            .setParameter("lawId", lawId)
            .getSingleResult();
        return count != null && count > 0;
    }
}
```

There is no update method. Updating a law means mutating the managed entity inside the transaction – the business layer loads it with `find()`, sets the fields, and JPA flushes. `cascade ALL` with `orphanRemoval` means the parameters follow automatically.

Step 7 — IHealthCommonService and HealthCommonService – six pass-throughs

Declare in the interface, implement as one-liners, and inject the DAO the same way IndicatorDeviationDao already is.

```
// field
@EJB
private FilteringLawDao filteringLawDao;

// interface + implementation
List<FilteringLaw> findFilteringLaws(String aircraftId) throws EntityNotFoundException;
FilteringLaw findFilteringLawById(String lawId);
void createFilteringLaw(String aircraftId, FilteringLaw law) throws EntityNotFoundException;
void deleteFilteringLaw(String aircraftId, String lawId) throws EntityNotFoundException;
boolean isFilteringLawUsedByConfiguration(String aircraftId, String lawName)
    throws EntityNotFoundException;
boolean isFilteringLawUsedByDeviation(String lawId);
```

Step 8 — IHealthCommonBusiness and HealthCommonBusiness – the four the client calls

getFilteringLaws already exists and returns id and name only. Extend it to fill the new fields, and add three more. Conversion happens here, as with the deviations.

```

@Override
public List<FilteringLawDto> getFilteringLaws(String aircraftId) throws EntityNotFoundException {
    List<FilteringLawDto> result = new ArrayList<>();
    for (FilteringLaw law : healthCommonService.findFilteringLaws(aircraftId)) {
        FilteringLawDto dto = new FilteringLawDto();
        dto.setId(law.getId());
        dto.setName(law.getName());
        dto.setOriginal(law.isOriginal());
        dto.setInUse(healthCommonService.isFilteringLawUsedByConfiguration(aircraftId, law.getName())
            || healthCommonService.isFilteringLawUsedByDeviation(law.getId()));
        for (ContextualParameter p : law.getContextualParameters()) {
            ContextualParameterDto pDto = new ContextualParameterDto();
            pDto.setId(p.getId());
            pDto.setName(p.getName());
            pDto.setMinValue(p.getMinValue());
            pDto.setMaxValue(p.getMaxValue());
            pDto.setThresholdOfVariabilityValue(p.getThresholdOfVariabilityValue());
            dto.getParameters().add(pDto);
        }
        result.add(dto);
    }
    result.sort((a, b) -> a.getName().compareTo(b.getName()));
    return result;
}

@Override
public void createFilteringLaw(String aircraftId, FilteringLawDto dto)
    throws EntityNotFoundException {
    FilteringLaw law = new FilteringLaw(UUID.randomUUID().toString());
    law.setName(dto.getName());
    law.setOriginal(false);
    applyParameters(law, dto);
    healthCommonService.createFilteringLaw(aircraftId, law);
}

@Override
public void updateFilteringLaw(FilteringLawDto dto) {
    FilteringLaw law = healthCommonService.findFilteringLawById(dto.getId());
    if (law == null) {
        return;
    }
    law.setName(dto.getName());
    law.getContextualParameters().clear(); // orphanRemoval deletes the old rows
    applyParameters(law, dto);
}

@Override
public void deleteFilteringLaw(String aircraftId, String lawId) throws EntityNotFoundException {
    healthCommonService.deleteFilteringLaw(aircraftId, lawId);
}

private void applyParameters(FilteringLaw law, FilteringLawDto dto) {
    for (ContextualParameterDto pDto : dto.getParameters()) {
        ContextualParameter p = new ContextualParameter(UUID.randomUUID().toString());
        p.setName(pDto.getName());
        p.setMinValue(pDto.getMinValue());
        p.setMaxValue(pDto.getMaxValue());
        p.setThresholdOfVariabilityValue(pDto.getThresholdOfVariabilityValue());

        law.getContextualParameters().add(p);
    }
}

```

Import `ContextualParameter` from `com.eurocopter.mgs.datamodel.health.monitoring` – the entity, not the enum of the same name in `health.common`. Both are in play in this feature, and picking the wrong one compiles in some places and fails in others.

`updateFilteringLaw` has no explicit save: the entity is managed inside the EJB transaction, so JPA flushes the changes. That is why there is no update in the DAO.

Client

Step 9 — HealthAlarmsPreferenceStore – four pass-throughs

Same shape as the deviation ones already there.

```
public List<FilteringLawDto> getFilteringLawDetails(Aircraft aircraft) {
    if (aircraft == null) {
        return new ArrayList<>();
    }
    try {
        return healthBusiness.getFilteringLaws(aircraft.getId());
    } catch (Exception e) {
        LOGGER.warn("Could not load filtering laws", e);
        return new ArrayList<>();
    }
}

public void createFilteringLaw(Aircraft aircraft, FilteringLawDto dto) {
    try {
        healthBusiness.createFilteringLaw(aircraft.getId(), dto);
    } catch (Exception e) {
        LOGGER.error("Could not create filtering law", e);
    }
}

public void updateFilteringLaw(FilteringLawDto dto) {
    try {
        healthBusiness.updateFilteringLaw(dto);
    } catch (Exception e) {
        LOGGER.error("Could not update filtering law", e);
    }
}

public void deleteFilteringLaw(Aircraft aircraft, String lawId) {
    try {
        healthBusiness.deleteFilteringLaw(aircraft.getId(), lawId);
    } catch (Exception e) {
        LOGGER.error("Could not delete filtering law", e);
    }
}
```

The existing getFilteringLawNames stays – the tree combo still uses it. The new method returns the full detail for the dialog.

Step 10 — FilteringLawComposite – new

mgs-health-app/.../preferences/healthalarms/FilteringLawComposite.java

Master table on top, detail form below: a name field and five parameter rows, each a parameter combo, min, max and stability. Selecting a read-only law disables the form.

```

package com.eurocopter.mgs.presentation.configuration.preferences.healthalarms;

public class FilteringLawComposite extends Composite {

    private static final int MAX_PARAMETERS = 5;

    private final HealthAlarmsPreferenceStore store;
    private final Aircraft aircraft;

    private final TableViewer lawViewer;
    private final Text nameText;
    private final Combo[] parameterCombos = new Combo[MAX_PARAMETERS];
    private final Text[] minTexts = new Text[MAX_PARAMETERS];
    private final Text[] maxTexts = new Text[MAX_PARAMETERS];
    private final Text[] stabilityTexts = new Text[MAX_PARAMETERS];
    private final Button createButton;
    private final Button updateButton;
    private final Button deleteButton;

    private FilteringLawDto selectedLaw;

    public FilteringLawComposite(Composite parent, HealthAlarmsPreferenceStore store,
        Aircraft aircraft) {
        super(parent, SWT.NONE);
        this.store = store;
        this.aircraft = aircraft;
        setLayout(new GridLayout(1, false));

        // ---- master: the laws of this aircraft
        Table lawTable = new Table(this, SWT.BORDER | SWT.FULL_SELECTION | SWT.V_SCROLL);
        lawTable.setHeaderVisible(true);
        lawTable.setLinesVisible(true);
        GridData lawData = new GridData(SWT.FILL, SWT.FILL, true, true);
        lawData.heightHint = 160;
        lawTable.setLayoutData(lawData);

        TableColumn nameColumn = new TableColumn(lawTable, SWT.LEFT);
        nameColumn.setText(Messages.CORE.getString("filteringLawColName"));
        nameColumn.setWidth(180);
        TableColumn paramColumn = new TableColumn(lawTable, SWT.LEFT);
        paramColumn.setText(Messages.CORE.getString("deviationCollawParameters"));
        paramColumn.setWidth(460);

        lawViewer = new TableViewer(lawTable);
        lawViewer.setContentProvider(ArrayContentProvider.getInstance());
        lawViewer.setLabelProvider(new LawLabelProvider());
        lawViewer.addSelectionChangedListener(event -> {
            IStructuredSelection selection = (IStructuredSelection) lawViewer.getSelection();
            showLaw((FilteringLawDto) selection.getFirstElement());
        });

        // ---- detail: name plus five parameter rows
        Composite form = new Composite(this, SWT.NONE);
        form.setLayout(new GridLayout(5, false));
        form.setLayoutData(new GridData(SWT.FILL, SWT.TOP, true, false));

        new Label(form, SWT.NONE).setText(Messages.CORE.getString("filteringLawColName"));
        nameText = new Text(form, SWT.BORDER);

```

```

GridData nameData = new GridData(SWT.FILL, SWT.CENTER, true, false);
nameData.horizontalSpan = 4;
nameText.setLayoutData(nameData);

new Label(form, SWT.NONE).setText("");
new Label(form, SWT.NONE).setText(Messages.CORE.getString("filteringLawColParameter"));
new Label(form, SWT.NONE).setText(Messages.CORE.getString("LearningThresholdMinValue"));
new Label(form, SWT.NONE).setText(Messages.CORE.getString("LearningThresholdMaxValue"));
new Label(form, SWT.NONE).setText(Messages.CORE.getString("filteringLawColStability"));

String[] parameterNames = parameterNames();
for (int i = 0; i < MAX_PARAMETERS; i++) {
    new Label(form, SWT.NONE).setText("CTXT" + (i + 1));
    parameterCombos[i] = new Combo(form, SWT.READ_ONLY | SWT.BORDER);
    parameterCombos[i].setItems(parameterNames);
    parameterCombos[i].setLayoutData(new GridData(SWT.FILL, SWT.CENTER, true, false));
    minTexts[i] = new Text(form, SWT.BORDER);
    minTexts[i].setLayoutData(new GridData(SWT.FILL, SWT.CENTER, true, false));
    maxTexts[i] = new Text(form, SWT.BORDER);
    maxTexts[i].setLayoutData(new GridData(SWT.FILL, SWT.CENTER, true, false));
    stabilityTexts[i] = new Text(form, SWT.BORDER);
    stabilityTexts[i].setLayoutData(new GridData(SWT.FILL, SWT.CENTER, true, false));
}

// ---- buttons
Composite buttons = new Composite(this, SWT.NONE);
buttons.setLayout(new GridLayout(3, false));
buttons.setLayoutData(new GridData(SWT.END, SWT.CENTER, true, false));
createButton = new Button(buttons, SWT.PUSH);
createButton.setText(Messages.CORE.getString("filteringLawCreate"));
createButton.addListener(SWT.Selection, e -> create());
updateButton = new Button(buttons, SWT.PUSH);
updateButton.setText(Messages.CORE.getString("filteringLawUpdate"));
updateButton.addListener(SWT.Selection, e -> update());
deleteButton = new Button(buttons, SWT.PUSH);
deleteButton.setText(Messages.CORE.getString("filteringLawDelete"));
deleteButton.addListener(SWT.Selection, e -> delete());

reload();
showLaw(null);
}

/** The parameter dropdown comes straight from the enum. */
private String[] parameterNames() {
    com.eurocopter.mgs.datamodel.health.common.ContextualParameter[] values =
        com.eurocopter.mgs.datamodel.health.common.ContextualParameter.values();
    String[] names = new String[values.length];
    for (int i = 0; i < values.length; i++) {
        names[i] = values[i].name();
    }
    Arrays.sort(names);
    return names;
}

private void reload() {
    lawViewer.setInput(store.getFilteringLawDetails(aircraft));
}
}

```

Continued in the next step: showLaw, create, update, delete, the label provider and the form readers.

Step 11 — FilteringLawComposite – the rest of the class

Paste these inside the same class, after reload().

```

/** Fill the form from the selected law, and lock it when the law is read-only. */
private void showLaw(FilteringLawDto law) {
    selectedLaw = law;
    boolean readOnly = law != null && (law.isOriginal() || law.isInUse());

    nameText.setText(law != null && law.getName() != null ? law.getName() : "");
    for (int i = 0; i < MAX_PARAMETERS; i++) {
        ContextualParameterDto p = law != null && i < law.getParameters().size()
            ? law.getParameters().get(i) : null;
        parameterCombos[i].setText(p != null && p.getName() != null ? p.getName() : "");
        minTexts[i].setText(p != null ? text(p.getMinValue()) : "");
        maxTexts[i].setText(p != null ? text(p.getMaxValue()) : "");
        stabilityTexts[i].setText(p != null ? text(p.getThresholdOfVariabilityValue()) : "");
    }
    setFormEnabled(!readOnly);
    updateButton.setEnabled(law != null && !readOnly);
    deleteButton.setEnabled(law != null && !readOnly);
}

private void setFormEnabled(boolean enabled) {
    nameText.setEnabled(enabled);
    for (int i = 0; i < MAX_PARAMETERS; i++) {
        parameterCombos[i].setEnabled(enabled);
        minTexts[i].setEnabled(enabled);
        maxTexts[i].setEnabled(enabled);
        stabilityTexts[i].setEnabled(enabled);
    }
}

private String text(Double value) {
    return value == null ? "" : String.valueOf(value);
}

private Double value(Text field) {
    String raw = field.getText().trim();
    if (raw.isEmpty()) {
        return null;
    }
    try {
        return Double.valueOf(raw);
    } catch (NumberFormatException e) {
        return null;
    }
}

/** Reads the form. Returns null and shows the reason when it is not valid. */
private FilteringLawDto readForm(String id) {
    String name = nameText.getText().trim();
    if (name.isEmpty()) {
        error("filteringLawErrorName");
        return null;
    }
    for (Object existing : (List<?>) lawViewer.getInput()) {
        FilteringLawDto other = (FilteringLawDto) existing;
        if (name.equals(other.getName()) && !other.getId().equals(id)) {
            error("filteringLawErrorDuplicate");
            return null;
        }
    }
}

```

```

    }

    FilteringLawDto dto = new FilteringLawDto();
    dto.setId(id);
    dto.setName(name);
    for (int i = 0; i < MAX_PARAMETERS; i++) {
        String parameter = parameterCombos[i].getText().trim();
        if (parameter.isEmpty()) {
            continue; // this row is simply unused
        }
        Double min = value(minTexts[i]);
        Double max = value(maxTexts[i]);
        if (min == null || max == null) {
            error("filteringLawErrorMinMax");
            return null;
        }
        if (min >= max) {
            error("filteringLawErrorMinMax");
            return null;
        }
        ContextualParameterDto p = new ContextualParameterDto();
        p.setName(parameter);
        p.setMinValue(min);
        p.setMaxValue(max);
        p.setThresholdOfVariabilityValue(value(stabilityTexts[i]));
        dto.getParameters().add(p);
    }
    if (dto.getParameters().isEmpty()) {
        error("filteringLawErrorNoParameter");
        return null;
    }
    return dto;
}

private void error(String key) {
    MessageDialog.openError(getShell(),
        Messages.CORE.getString("filteringLawErrorTitle"),
        Messages.CORE.getString(key));
}

private void create() {
    FilteringLawDto dto = readForm(null);
    if (dto == null) {
        return;
    }
    store.createFilteringLaw(aircraft, dto);
    reload();
    showLaw(null);
}

private void update() {
    if (selectedLaw == null) {
        return;
    }
    FilteringLawDto dto = readForm(selectedLaw.getId());
    if (dto == null) {
        return;
    }
}

```

```

        store.updateFilteringLaw(dto);
        reload();
        showLaw(null);
    }

    private void delete() {
        if (selectedLaw == null) {
            return;
        }
        boolean confirmed = MessageDialog.openConfirm(getShell(),
            Messages.CORE.getString("filteringLawDeleteTitle"),
            MessageFormat.format(Messages.CORE.getString("filteringLawDeleteConfirm"),
                selectedLaw.getName()));
        if (!confirmed) {
            return;
        }
        store.deleteFilteringLaw(aircraft, selectedLaw.getId());
        reload();
        showLaw(null);
    }

    private final class LawLabelProvider extends LabelProvider implements ITableLabelProvider {

        @Override
        public Image getColumnImage(Object element, int column) {
            return null;
        }

        @Override
        public String getColumnText(Object element, int column) {
            FilteringLawDto law = (FilteringLawDto) element;
            if (column == 0) {
                return law.getName();
            }
            StringBuilder text = new StringBuilder();
            for (ContextualParameterDto p : law.getParameters()) {
                if (text.length() > 0) {
                    text.append(" | ");
                }
                text.append(p.getName())
                    .append(", min=").append(p.getMinValue())
                    .append(", max=").append(p.getMaxValue());
                if (p.getThresholdOfVariabilityValue() != null) {
                    text.append(", S=").append(p.getThresholdOfVariabilityValue());
                }
            }
            return text.toString();
        }
    }
}

```

Step 12 — FilteringLawDialog – new

mgs-health-app/.../preferences/healthalarms/FilteringLawDialog.java

Follows BatchMarkerCreationDialog exactly: extends Dialog, hosts the composite, adapts it with a FormToolkit. Close only, since Create, Update and Delete act immediately.

```

public class FilteringLawDialog extends Dialog {

    private final HealthAlarmsPreferenceStore store;
    private final Aircraft aircraft;
    private final FormToolkit toolkit;

    public FilteringLawDialog(HealthAlarmsPreferenceStore store, Aircraft aircraft) {
        super(PlatformUI.getWorkbench().getDisplay().getActiveShell());
        this.store = store;
        this.aircraft = aircraft;
        this.toolkit = new FormToolkit(PlatformUI.getWorkbench().getDisplay());
        setShellStyle(getShellStyle() | SWT.RESIZE);
    }

    @Override
    protected void configureShell(Shell shell) {
        super.configureShell(shell);
        shell.setText(Messages.CORE.getString("filteringLawDialogTitle"));
        shell.setSize(900, 560);
    }

    @Override
    protected Control createDialogArea(Composite parent) {
        parent.setBackground(Display.getDefault().getSystemColor(SWT.COLOR_WHITE));
        parent.setBackgroundMode(SWT.INHERIT_FORCE);
        FilteringLawComposite composite = new FilteringLawComposite(parent, store, aircraft);
        composite.setLayoutData(new GridData(SWT.FILL, SWT.FILL, true, true));
        toolkit.adapt(composite);
        toolkit.adapt(parent);
        ColorUtils.forceBackgroundColor(parent);
        return composite;
    }

    @Override
    protected void createButtonsForButtonBar(Composite parent) {
        toolkit.adapt(parent);
        createButton(parent, IDialogConstants.CLOSE_ID,
            Messages.CORE.getMnemonicEscapedString("Close"), true);
        ColorUtils.forceBackgroundColor(parent);
    }

    @Override
    protected void buttonPressed(int buttonId) {
        if (buttonId == IDialogConstants.CLOSE_ID) {
            close();
        }
    }
}

```

Check that the key "Close" exists in CoreMessages; if not, use Cancel_Button_Label, which BatchMarkerCreationDialog already uses.

Step 13 — HealthAlarmsComposite – the button

Add a button row under the tab folder, in the constructor after the tabs are created.

```

Composite lawButtonBar = new Composite(this, SWT.NONE);
lawButtonBar.setLayout(new GridLayout(1, false));
lawButtonBar.setLayoutData(new GridData(SWT.END, SWT.CENTER, true, false));
Button manageLawsButton = new Button(lawButtonBar, SWT.PUSH);
manageLawsButton.setText(Messages.CORE.getString("filteringLawManageButton"));
manageLawsButton.addListener(SWT.Selection, e -> {
    if (selectedAircraft == null) {
        return;
    }
    new FilteringLawDialog(healthAlarmsPreferenceStore, selectedAircraft).open();
    reloadAll(); // laws may have changed, so refresh the tree combo
});

```

Step 14 — CoreMessages.properties – and every language file

New keys. The min and max labels reuse the tree's existing ones.

```

filteringLawManageButton=Manage Filtering Laws
filteringLawDialogTitle=Manage Filtering Laws
filteringLawColName=Name
filteringLawColParameter=Parameter
filteringLawColStability=Stability
filteringLawCreate=Create
filteringLawUpdate=Update
filteringLawDelete=Delete
filteringLawDeleteTitle=Delete filtering law
filteringLawDeleteConfirm=Are you sure you want to delete the filtering law {0}
filteringLawErrorTitle=Filtering law
filteringLawErrorName=The name is mandatory.
filteringLawErrorDuplicate=A filtering law with this name already exists.
filteringLawErrorMinMax=Each parameter needs a minimum and a maximum, and the minimum must be \
smaller than the maximum.
filteringLawErrorNoParameter=A filtering law needs at least one contextual parameter.

```

Build and test

#	Action
1	Run the DDL, then rebuild mgs-jpa-model, mgs-services-interfaces, mgs-services-impl, mgs-health-business, then the client. Redeploy.
2	Open Health, pick an aircraft, press Manage Filtering Laws – the existing laws are listed with their parameters
3	Select a law already used by an indicator – the form greys out and Update and Delete are disabled
4	Type a new name, pick NR_MAIN_ROTOR_SPEED, min 500, max 2500, press Create – it appears in the list
5	Select it, add a second parameter, press Update – the list shows both
6	Press Delete, confirm – it goes, and SELECT * FROM public.contextual_parameter shows its rows gone too
7	Close, then check the Filtering Law dropdown in the tree includes the new law
8	Try Create with min 2500 and max 500 – rejected with a message

Item 6 is the one that proves orphanRemoval is working. If the parameter rows survive the delete, the cascade is not firing and the law is being removed outside the managed set.

Still open after this

Rename of a law in use is blocked rather than propagated, because indicator_configuration stores the law name as a string. If the client wants rename to work on an in-use law, updateFilteringLaw must also rewrite every matching indicator_configuration row, inside the same transaction. Raise it with Benoit along with the ORIGINAL column from Step 1.